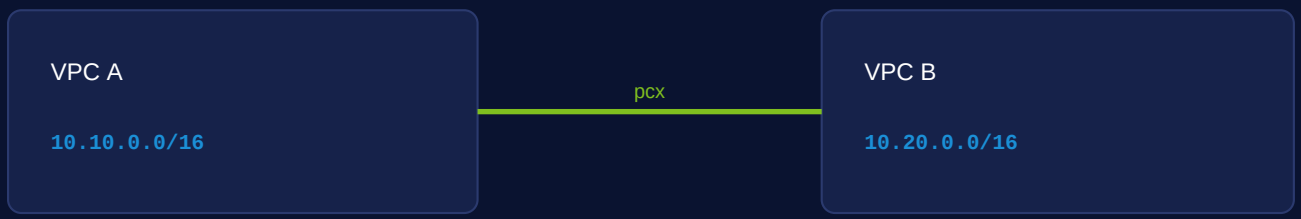
AWS NETWORKING / PROJECT 01

VPC Peering

Connecting two VPCs and proving private connectivity

Project report: EC2, custom route tables, security groups and a peering connection

2 VPCs	3 subnets	1 peering connection	2 EC2 instances



PROJECT DETAILS

EWB Courses | AWS Architect Associate

B.Tech Information Technology, BIET Jhansi

Programme: EWB Courses, AWS Architect Associate (Project 01, Oct 2026)

Region: Asia Pacific (Sydney), ap-southeast-2 | Completed: 18 October 2026

LEARN. ELEVATE. EMPOWER.

Contents

1. Objective . . . 3
2. Architecture and address plan . . . 3
3. Resource inventory . . . 4
4. Implementation . . . 5
5. Security design . . . 12
6. Testing and verification . . . 13
7. Forward path and return path . . . 14
8. Key concepts and questions answered . . . 14
9. Troubleshooting checklist . . . 15
10. Cost, quotas and limits . . . 15
11. Choosing peering or an alternative . . . 16
12. AWS CLI equivalent . . . 16
13. Hardening and next steps . . . 17
14. Cleanup plan . . . 18
15. What I learned . . . 18
Appendix A: Glossary . . . 19
Appendix B: Further reading . . . 19

Project at a glance

2	3	1	2
VPCs	subnets	peering connection (Active)	t3.micro EC2 instances

Item	Detail
Region	Asia Pacific (Sydney), ap-southeast-2
VPC A	ewb-vpc-a, 10.10.0.0/16: one public and one private subnet, the only VPC with an internet gateway
VPC B	ewb-vpc-b, 10.20.0.0/16: one private subnet, no internet gateway
Peering connection	ewb-a-b-peer, pcx-0ee8e4be0ede9ba99, status Active
Compute	ewb-ec2-a (VPC A) and ewb-ec2-b (VPC B), t3.micro, ap-southeast-2a, no public IPv4 address
Security	Inbound rules limited to /32 sources; nothing opened to 0.0.0.0/0
Verification	Six tests: baseline, forward path, reverse path, private SSH, isolation, controlled failure

1. Objective

The goal of this project was to connect two separate AWS Virtual Private Clouds (VPCs) with a VPC peering connection so that workloads in each VPC can talk to each other over private IPv4 addresses, without using the public internet. The build also had to show that each route and security rule has a purpose: non-overlapping CIDR ranges, explicit route table associations, forward and return routes, and tightly scoped inbound rules.

What this report demonstrates:

- Planning non-overlapping VPC and subnet CIDR ranges.
- Creating public and private subnets with explicit route table associations.
- Requesting, accepting and activating a peering connection, then adding routes on both sides.
- Launching EC2 instances in the two VPCs and verifying private connectivity.

2. Architecture and address plan

VPC A (`10.10.0.0/16`) holds a public and a private subnet and is the only VPC with an internet gateway. VPC B (`10.20.0.0/16`) has a single private subnet and no internet gateway. A peering connection links the two VPCs, so the private subnets reach each other directly using private IP addresses.

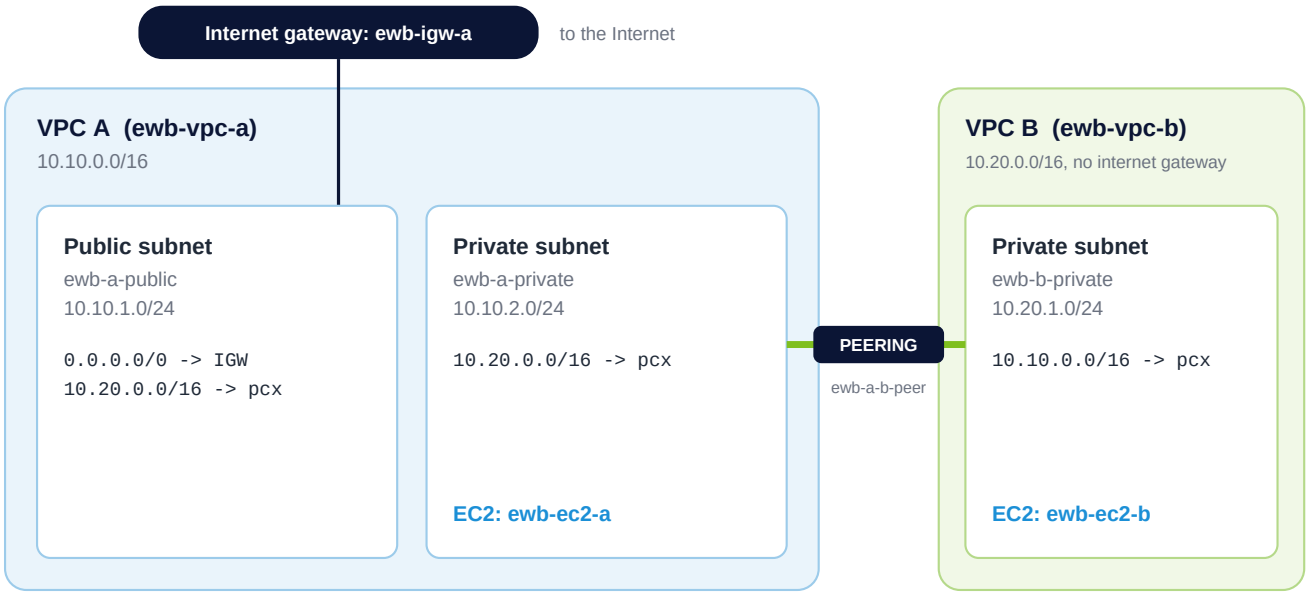


Figure 1. Architecture of the project (conceptual). Peering does not give VPC B internet access through VPC A.

Address plan

Resource (Name tag)	Placement	IPv4 CIDR
ewb-vpc-a	VPC A	`10.10.0.0/16`
ewb-a-public	VPC A, public subnet	`10.10.1.0/24`
ewb-a-private	VPC A, private subnet	`10.10.2.0/24`
ewb-vpc-b	VPC B	`10.20.0.0/16`
ewb-b-private	VPC B, private subnet	`10.20.1.0/24`

The two VPC ranges do not overlap, which is a hard requirement: AWS refuses to create a peering connection between VPCs with overlapping CIDRs. Every /24 subnet sits inside its own VPC range. Each /24 gives 251 usable addresses, because AWS reserves five addresses in every subnet.

Design choices and why they matter

Design choice	What was done	Why it matters
CIDR plan	10.10.0.0/16 and 10.20.0.0/16	Distinct ranges are mandatory for peering and leave room for more subnets later.
VPC creation mode	"VPC only" in the wizard	Every subnet, route table and gateway is created by hand, so each piece is visible and understood.
Route tables	Custom tables with explicit associations	A subnet with no explicit association silently uses the main route table (visible in Figure 7).
Internet exposure	Only the VPC A public subnet routes to the IGW	Private workloads cannot be reached from the internet; VPC B has no gateway at all.
Security groups	/32 sources, no 0.0.0.0/0	Least privilege: only named lab hosts may connect.
Placement	Both instances in ap-southeast-2a	Same-AZ traffic over a peering connection carries no data transfer charge.

3. Resource inventory

The table lists the resources created in the **Asia Pacific (Sydney) ap-southeast-2** Region, with the IDs shown in the AWS console.

Resource	Name	ID / detail
VPC A	ewb-vpc-a	`vpc-0e13b1275c4404f86 (10.10.0.0/16)`
VPC B	ewb-vpc-b	`vpc-0b570eaf09596cbfa (10.20.0.0/16)`
Subnet	ewb-a-public	`subnet-093184ce1cf20194c (10.10.1.0/24)`
Subnet	ewb-a-private	`subnet-0c3c48d885ebe... (10.10.2.0/24)`
Subnet	ewb-b-private	`subnet-09aac6debd58277e4 (10.20.1.0/24)`
Internet gateway	ewb-igw-a	`igw-083541cabc1ddaac6, attached to ewb-vpc-a`
Route table	ewb-rt-a-public	`rtb-0f1d18e9fea198201, associated with ewb-a-public`
Route table	ewb-rt-a-private	`Created in VPC A for ewb-a-private`
Route table	ewb-rt-b-private	`rtb-0a50e327a9e62155e, associated with ewb-b-private`
Peering connection	ewb-a-b-peer	`pcx-0ee8e4be0ede9ba99, status Active`
EC2 instance	ewb-ec2-a	`i-0b2fa2f3b7306a262, t3.micro, ap-southeast-2a`
EC2 instance	ewb-ec2-b	`i-08a657c4b1065e30b, t3.micro, ap-southeast-2a`

Build order

Each resource depends on the one before it, so the project was built in the order below. Skipping ahead, for example adding a peer route before the peering connection is accepted, is not possible or has no effect.

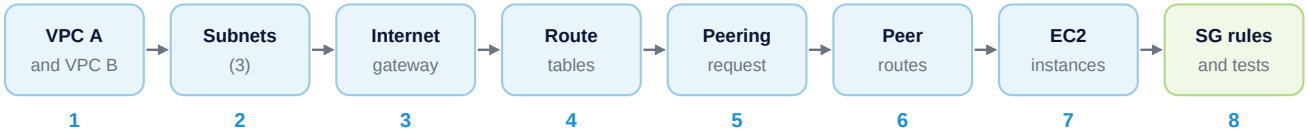


Diagram 1. Build order used for the project. Steps 1 to 8 in the next section follow the same sequence.

4. Implementation

Step 1: Create VPC A

In the VPC console I chose *Create VPC* with **VPC only**, so that every subnet and route could be controlled manually. The VPC was named `ewb-vpc-a` with the IPv4 CIDR `10.10.0.0/16`, no IPv6 CIDR and default tenancy. The VPC reached the Available state.

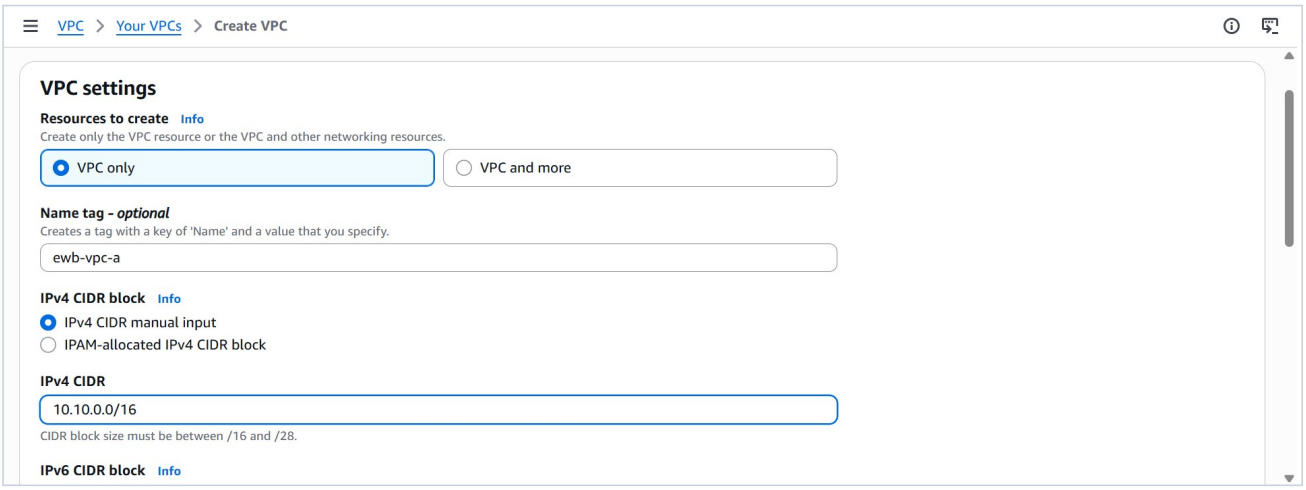


Figure 2. Create VPC wizard: VPC only, name ewb-vpc-a, IPv4 CIDR 10.10.0.0/16.

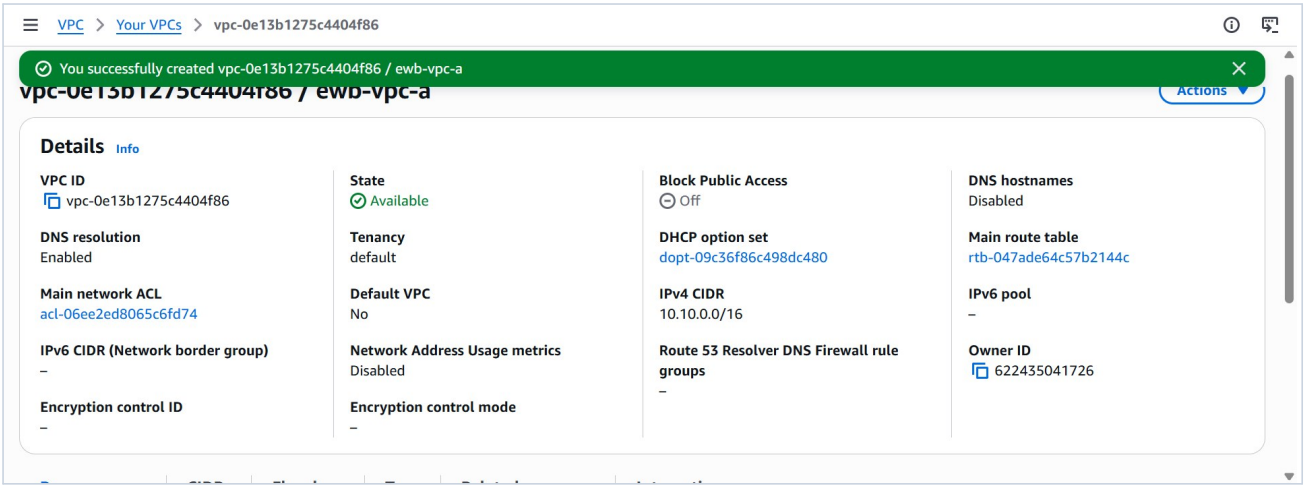


Figure 3. VPC ewb-vpc-a (vpc-0e13b1275c4404f86) created successfully and in the Available state.

> **What to notice:** AWS also created a main route table (rtb-047ade64c57b2144c) and a default network ACL for the VPC. This project does not rely on the main route table; it uses custom tables with explicit associations instead.

Step 2: Create the subnets in VPC A

Two subnets were created in VPC A: `ewb-a-public` (10.10.1.0/24) and `ewb-a-private` (10.10.2.0/24). The subnet associations screen in Figure 7 lists both subnets with their CIDRs.

Step 3: Attach the internet gateway

An internet gateway named `ewb-igw-a` was created and attached to ewb-vpc-a. An internet gateway belongs to the VPC and is used as a route target in the route table, so attaching it alone does not give a subnet internet access.

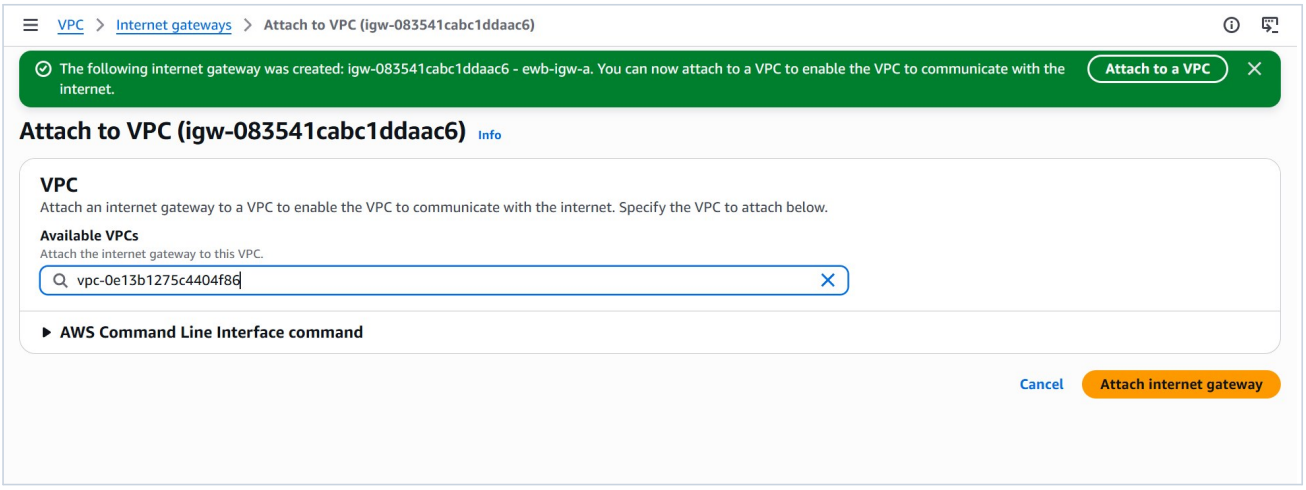


Figure 4. Attaching internet gateway ewb-igw-a (igw-083541cabc1ddaac6) to vpc-0e13b1275c4404f86.

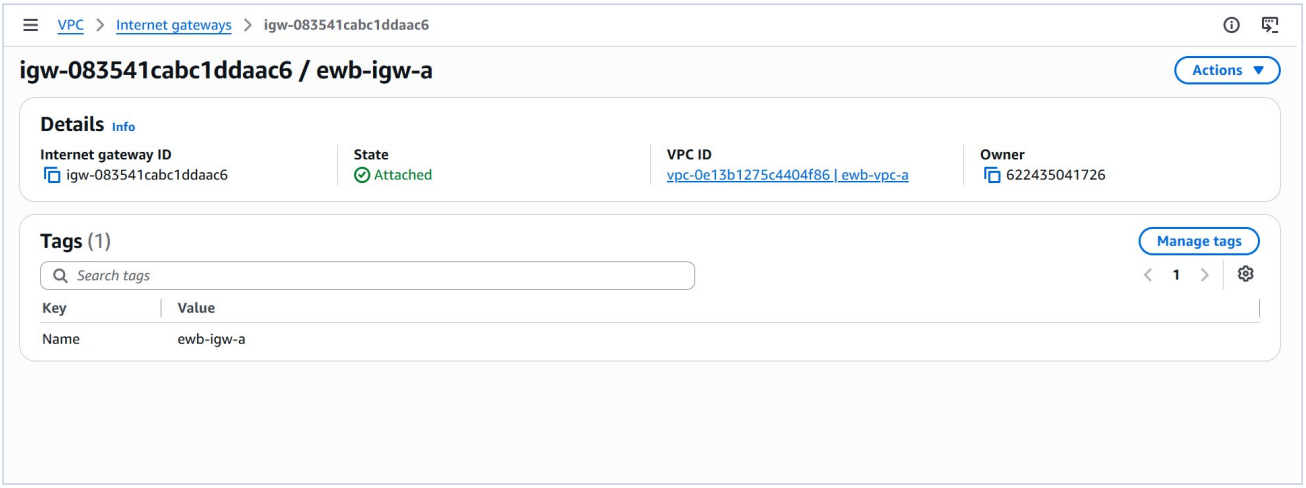


Figure 5. Internet gateway ewb-igw-a is in the Attached state, linked to ewb-vpc-a.

Step 4: Create the route tables in VPC A

A custom route table `ewb-rt-a-public` was created for VPC A, and ewb-a-public was explicitly associated with it. A second route table, `ewb-rt-a-private`, was created for the private subnet. It keeps only the local route and has no internet default route, so the private subnet stays private.

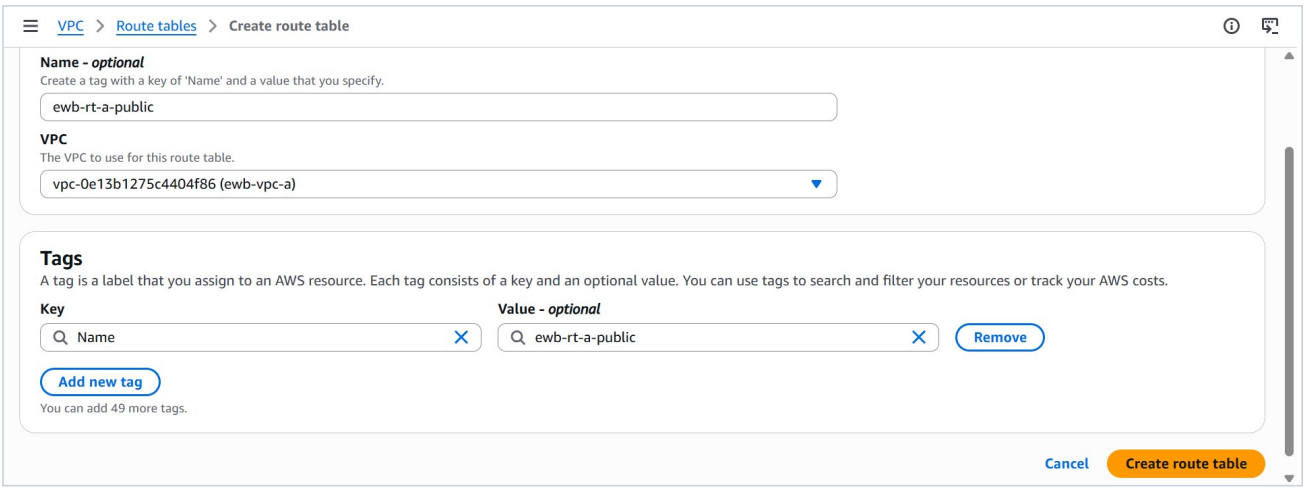


Figure 6. Creating route table ewb-rt-a-public in vpc-0e13b1275c4404f86 (ewb-vpc-a).

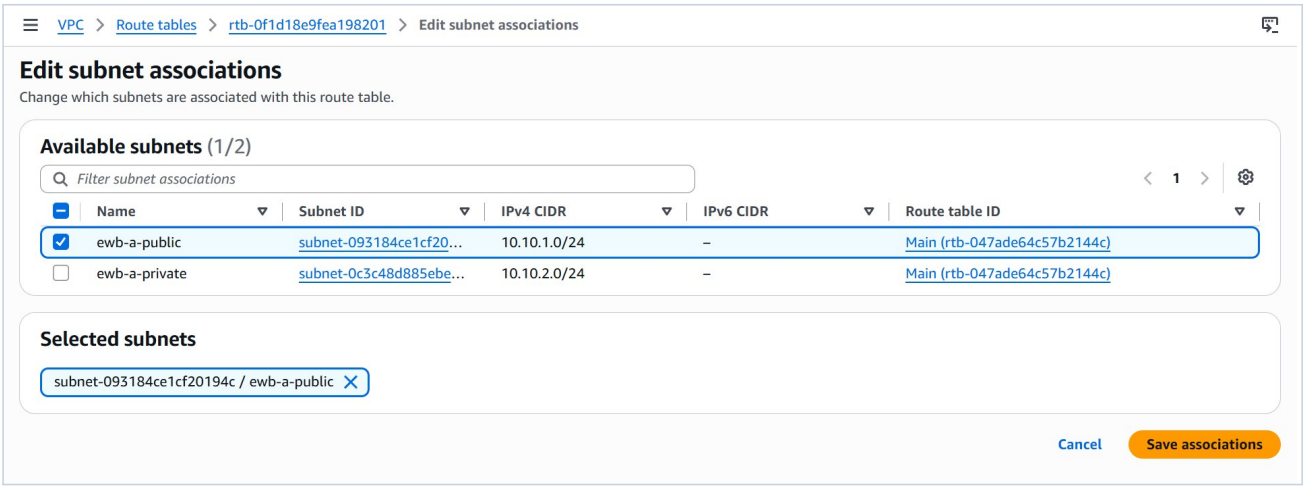


Figure 7. Explicit subnet association: ewb-a-public (10.10.1.0/24) selected for ewb-rt-a-public.

What to notice: In Figure 7 both subnets still show "Main (rtb-047ade64c57b2144c)" in the Route table ID column. That is the implicit association every new subnet starts with. Saving the association replaces it for ewb-a-public only.

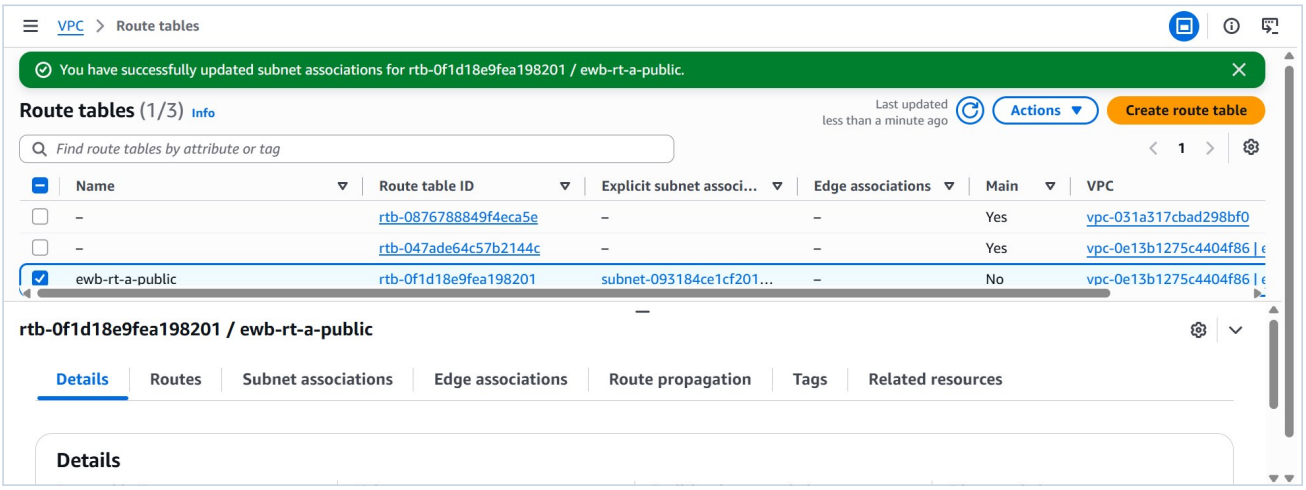


Figure 8. Subnet associations updated successfully; ewb-rt-a-public is now associated with subnet-093184ce1cf20...

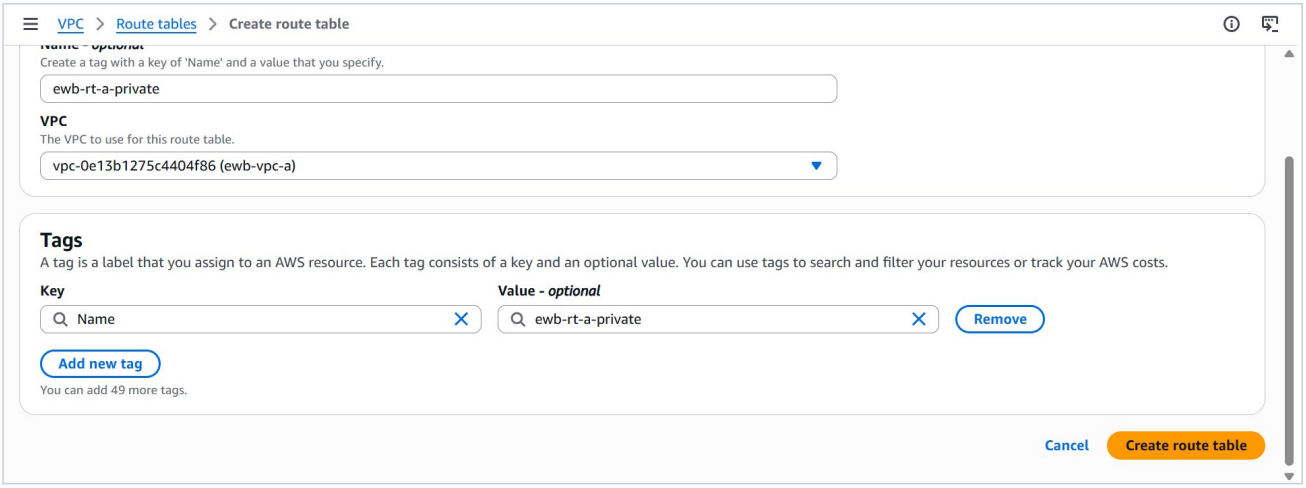


Figure 9. Creating route table ewb-rt-a-private in ewb-vpc-a.

Step 5: Create VPC B, its subnet and route table

VPC B (`ewb-vpc-b`, 10.20.0.0/16) was created with no internet gateway. Inside it, the private subnet `ewb-b-private` (10.20.1.0/24) and the route table `ewb-rt-b-private` were created, and the subnet was explicitly associated with the route table.

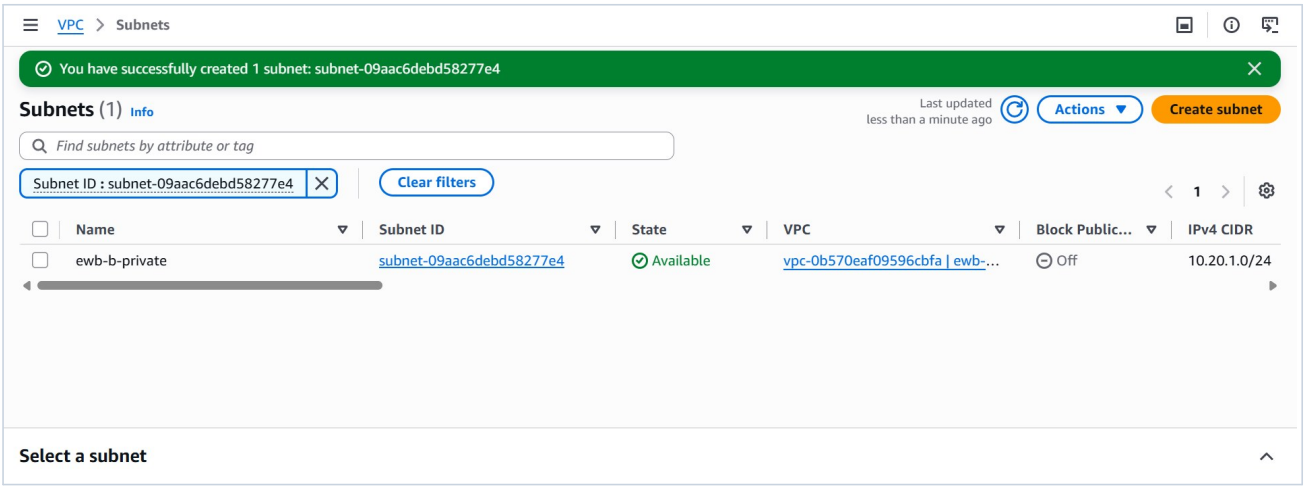


Figure 10. Subnet ewb-b-private (subnet-09aac6debd58277e4, 10.20.1.0/24) created and Available in VPC B.

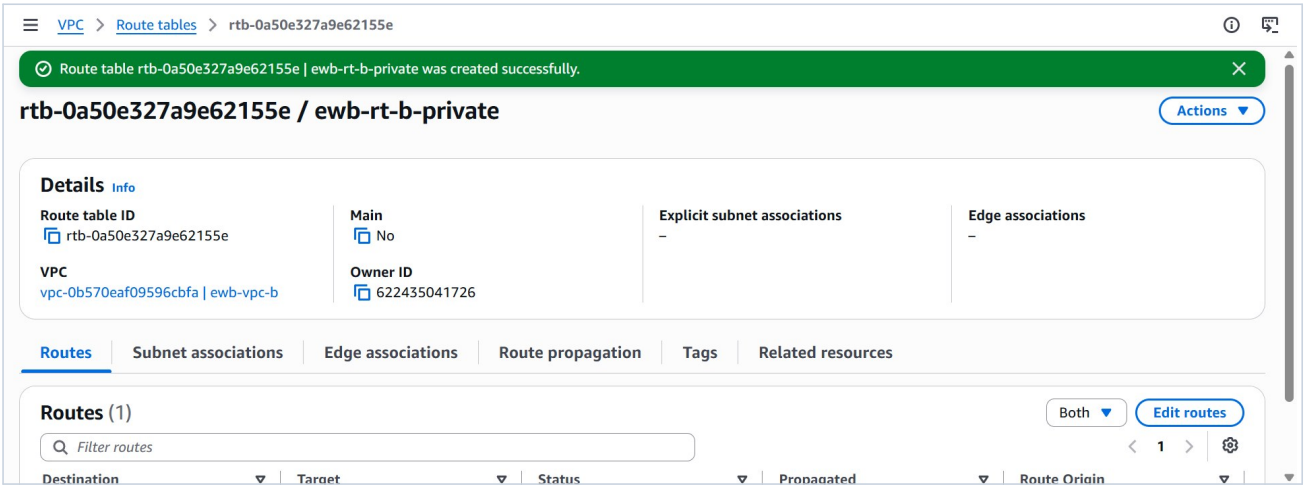


Figure 11. Route table ewb-rt-b-private (rtb-0a50e327a9e62155e) created in ewb-vpc-b.

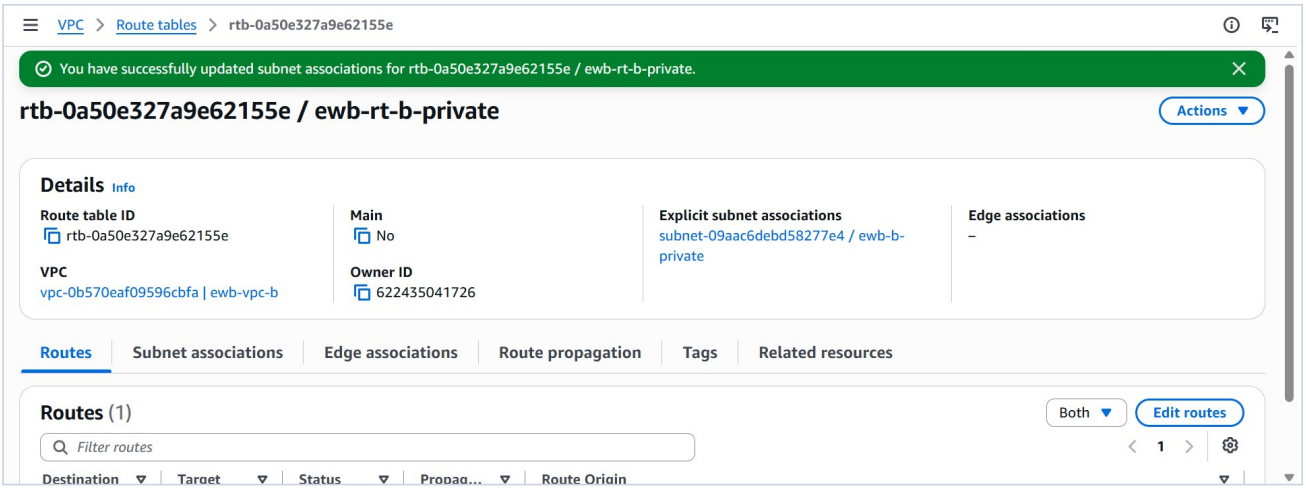


Figure 12. Subnet association updated: subnet-09aac6debd58277e4 / ewb-b-private is now explicitly associated with ewb-rt-b-private.

Step 6: Request and accept the peering connection

A peering connection named `ewb-a-b-peer` was requested with ewb-vpc-a as the requester and ewb-vpc-b as the accepter. Both VPCs are in the same account and Region. The request first showed *Pending acceptance*. Even within one account, the request must be accepted manually. After acceptance the status changed to **Active**, and the requester CIDR (10.10.0.0/16) and accepter CIDR (10.20.0.0/16) were both shown.

Even inside one account and Region, the request is raised by one VPC and accepted for the other.

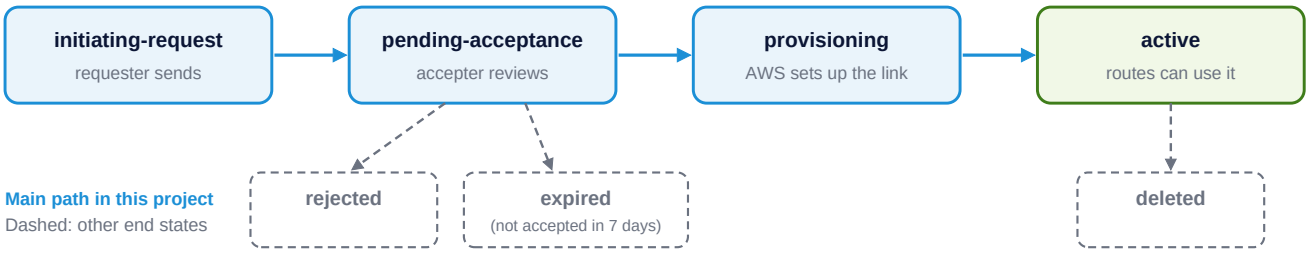


Diagram 2. Life cycle of a peering connection. The project followed the main path from request to Active.

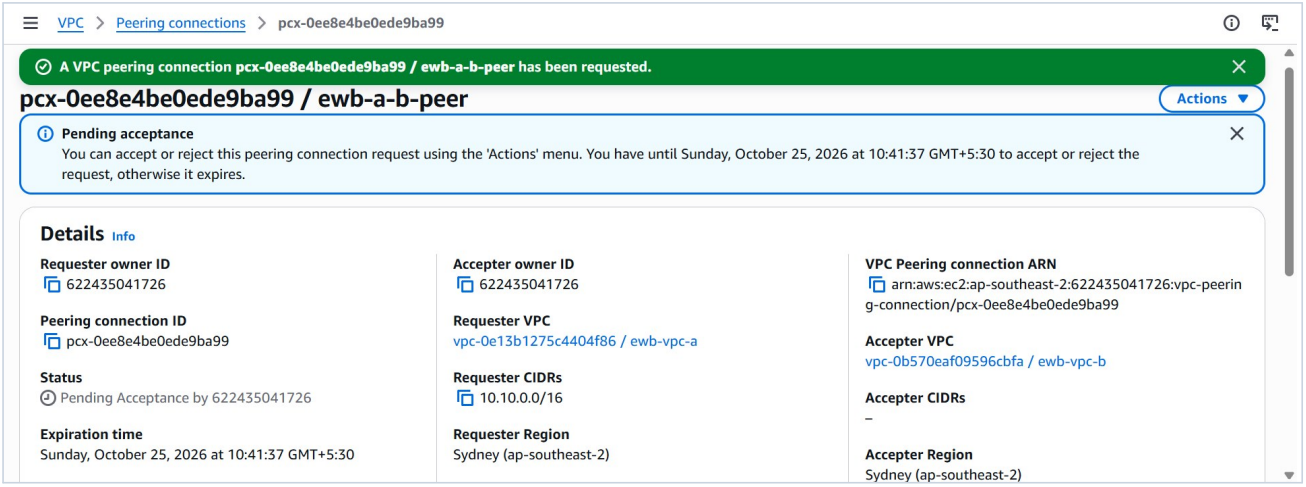


Figure 13. Peering connection pcx-0ee8e4be0ede9ba99 (ewb-a-b-peer) requested: status Pending acceptance.

What to notice: Figure 13 shows an expiry time for the request (unaccepted requests lapse after one week), and the Accepter CIDRs field is still empty. After acceptance (Figure 14) the expiry disappears and both CIDRs are listed.

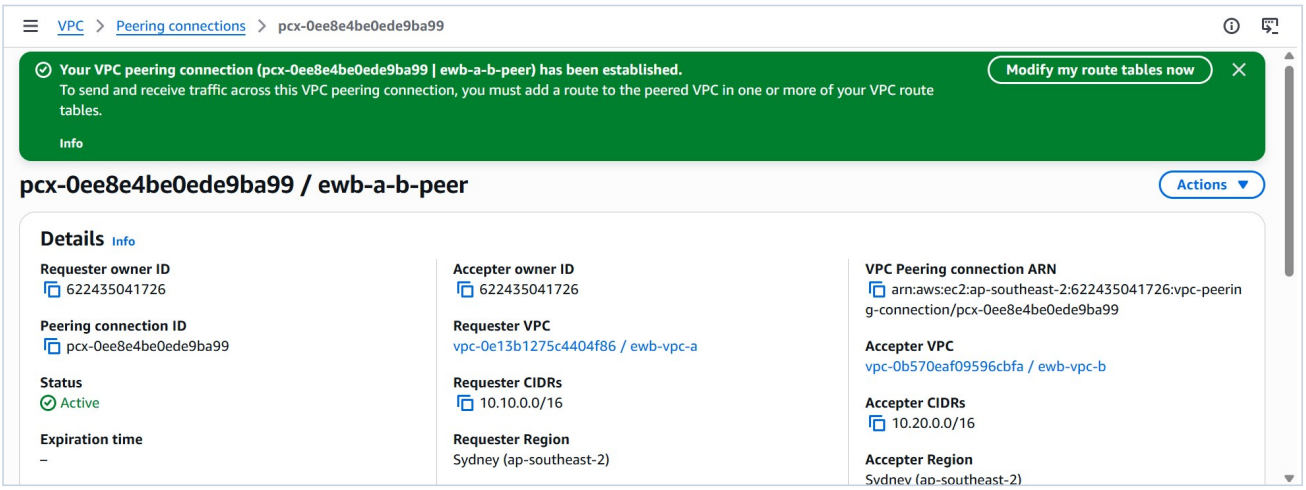


Figure 14. Peering connection ewb-a-b-peer established and Active, with requester 10.10.0.0/16 and accepter 10.20.0.0/16.

Important: an Active peering connection does not by itself let traffic flow. It only creates the link. Routes in both VPCs and the security group rules decide whether traffic can use it.

Step 7: Add routes on both sides

Each side needs a route to the other VPC, because a reply needs a way back as well as a way out. The routes below were configured, with `pcx-0ee8e4be0ede9ba99` as the target for the peer CIDR.

Route table	Destination	Target	Purpose
ewb-rt-a-private	10.10.0.0/16	local	VPC A internal traffic
ewb-rt-a-private	10.20.0.0/16	pcx (ewb-a-b-peer)	Forward path from VPC A to VPC B
ewb-rt-b-private	10.20.0.0/16	local	VPC B internal traffic
ewb-rt-b-private	10.10.0.0/16	pcx (ewb-a-b-peer)	Return path from VPC B to VPC A
ewb-rt-a-public	10.10.0.0/16	local	VPC A internal traffic
ewb-rt-a-public	0.0.0.0/0	igw (ewb-igw-a)	Internet access for the public subnet
ewb-rt-a-public	10.20.0.0/16	pcx (ewb-a-b-peer)	Public subnet to VPC B

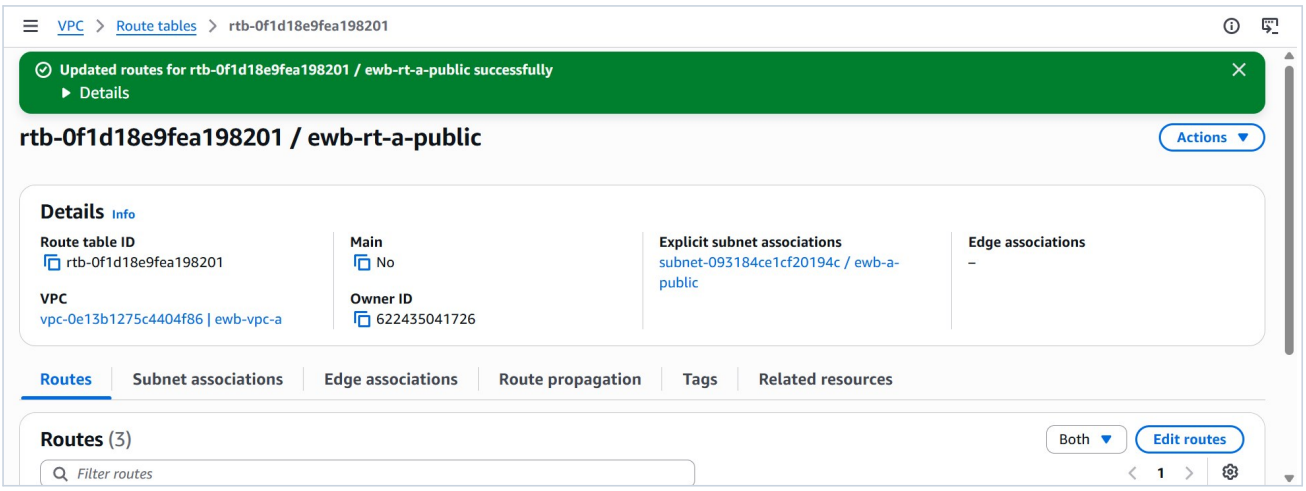


Figure 15. Routes updated successfully for ewb-rt-a-public (rtb-0f1d18e9fea198201); the route table now has 3 routes.

The more specific `10.20.0.0/16` route is chosen for traffic to VPC B instead of the broader internet default route, because AWS always picks the most specific matching route. The table below applies that rule to a few sample destinations (the host addresses are illustrative).

Destination IP	Route table used	Matching routes	Route chosen
10.10.2.25 (own VPC)	ewb-rt-a-public	10.10.0.0/16 (local), 0.0.0.0/0 (igw)	local, because /16 is more specific than /0
10.20.1.15 (VPC B)	ewb-rt-a-private	10.20.0.0/16 (pcx)	pcx
10.20.1.15 (VPC B)	ewb-rt-a-public	10.20.0.0/16 (pcx), 0.0.0.0/0 (igw)	pcx, because /16 is more specific than /0
8.8.8.8 (internet)	ewb-rt-a-public	0.0.0.0/0 (igw)	igw
8.8.8.8 (internet)	ewb-rt-a-private	none	No route: traffic is dropped, so the subnet stays private

Table 1. Longest-prefix-match examples for the route tables built in this project.

Step 8: Launch the EC2 instances

Two `t3.micro` instances were launched in Availability Zone ap-southeast-2a: `ewb-ec2-a` for VPC A and `ewb-ec2-b` for VPC B. Both reached the Running state and passed all 3 of 3 status checks.

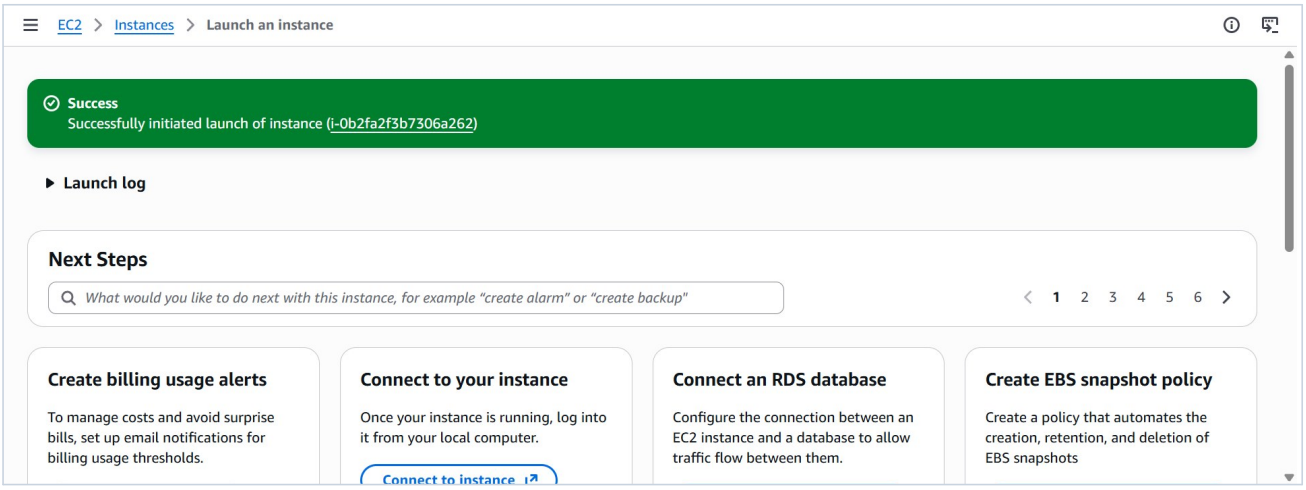


Figure 16. EC2 launch initiated successfully for instance i-0b2fa2f3b7306a262.

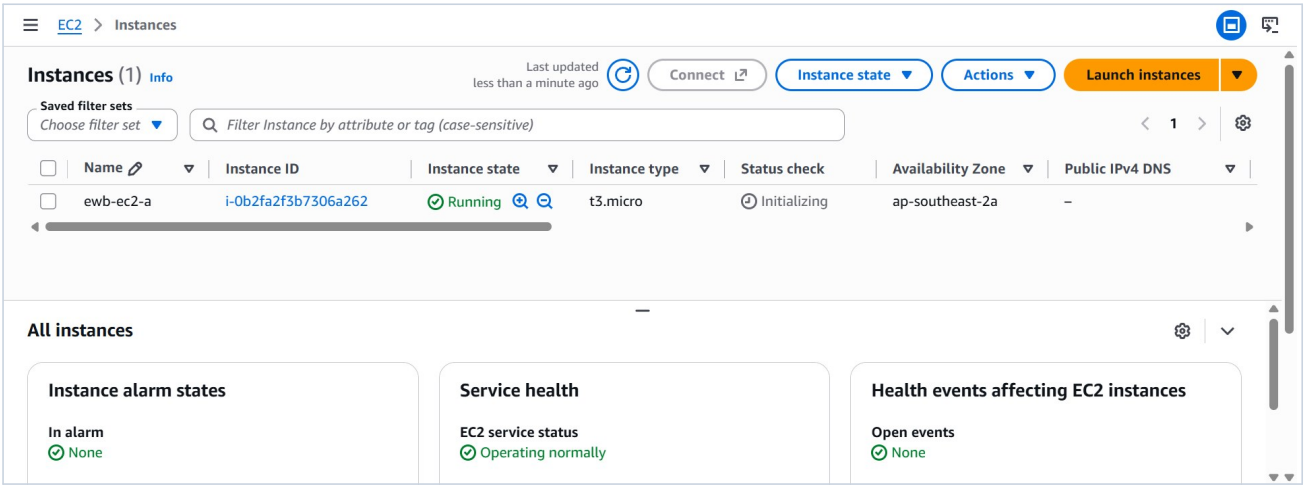


Figure 17. ewb-ec2-a (i-0b2fa2f3b7306a262) running; status checks initializing.

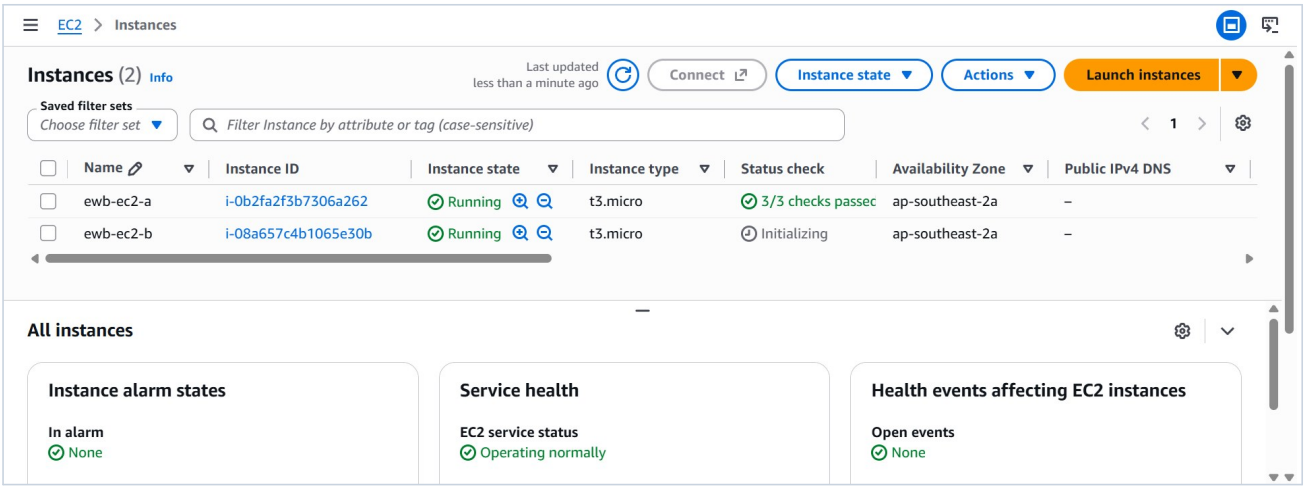


Figure 18. Second instance ewb-ec2-b (i-08a657c4b1065e30b) launched; ewb-ec2-a now shows 3/3 checks passed.

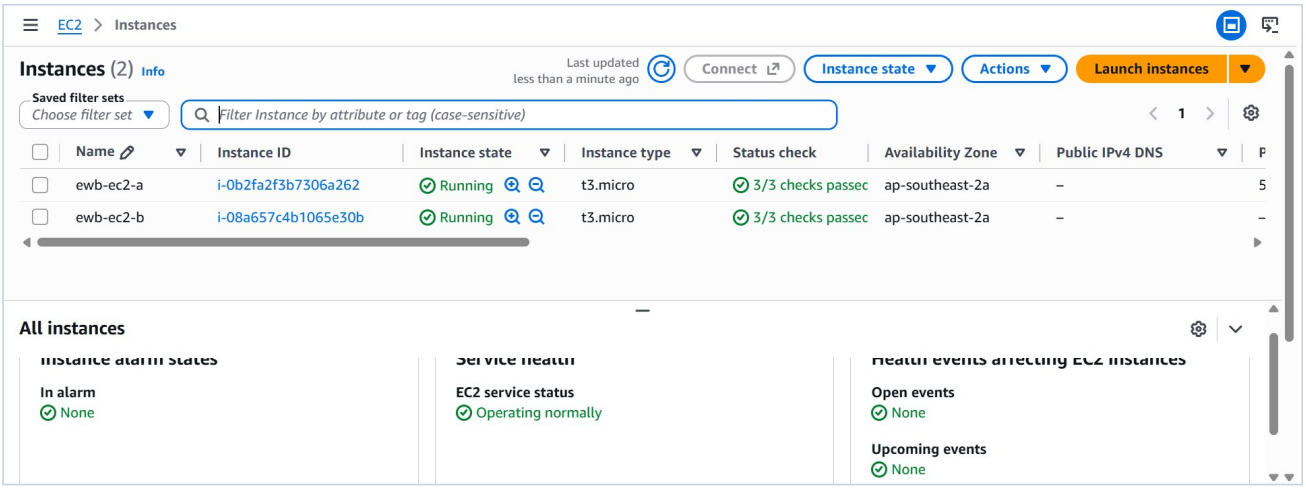


Figure 19. Both instances Running with 3/3 status checks passed, ready for connectivity testing.

> **What to notice:** The Public IPv4 DNS column in Figures 17 to 19 shows a dash for both instances. Neither has a public address, which is the evidence used later for the isolation test.

5. Security design

Security groups act as stateful firewalls, so replies to permitted traffic are allowed automatically. The inbound rules follow the principle of least privilege: only the specific lab hosts are allowed, and nothing is opened to `0.0.0.0/0`.

Security group	Type / port	Source
ewb-sg-app-a (VPC A)	Custom ICMP IPv4, Echo request	Private IP of the instance in VPC B (/32)
ewb-sg-app-a (VPC A)	SSH / TCP 22	Private IP of the administration host (/32)
ewb-sg-app-b (VPC B)	Custom ICMP IPv4, Echo request	Private IP of the instance in VPC A (/32)
ewb-sg-app-b (VPC B)	SSH / TCP 22	Private IP of the administration host (/32)

Both application instances are private: they have no public IPv4 address, and the subnets are not routed to an internet gateway except the public subnet in VPC A. The key pair stays on my own computer and is never copied to any instance.

Layers a packet must pass

A packet has to pass every layer in order. An Active peering connection satisfies only the first condition.

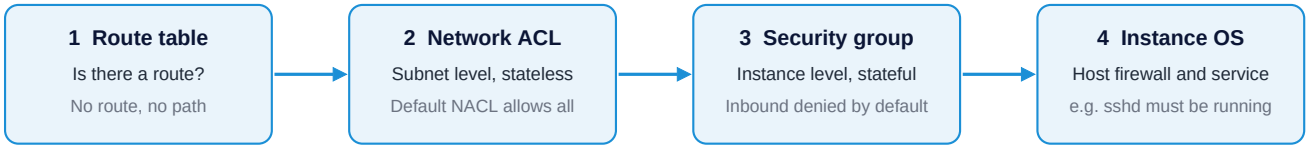


Diagram 3. Four independent checks decide whether traffic arrives. Each one can block traffic on its own.

This is why the controlled failure test in section 6 is useful: removing one security group rule stops the ping even though layer 1 (route and peering) is perfect.

6. Testing and verification

Peering is considered successful only if traffic actually flows, so the tests below check both directions and also include a controlled failure. All tests use private destination IP addresses.

Test	Where / command	Expected result
01 Baseline	ewb-ec2-a, before peering routes: `ping -c 4 <private IP of ewb-ec2-b>`	No replies
02 Forward path	ewb-ec2-a, after routes: `ping -c 4 <private IP of ewb-ec2-b>`	4 replies, 0% packet loss
03 Reverse path	ewb-ec2-b, after routes: `ping -c 4 <private IP of ewb-ec2-a>`	4 replies, 0% packet loss
04 Private SSH	SSH to ewb-ec2-b through the bastion path	Shell on ewb-ec2-b
05 Isolation	EC2 instance details of both private hosts	No public IPv4 address
06 Controlled failure	Remove only the ICMP Echo rule on ewb-sg-app-b, ping again, then restore the exact rule	Replies stop, then resume

Explanation of the controlled failure: removing the Echo request rule changes only the security layer, while the route and peering stay in place. The ping fails, which proves that security groups are evaluated on top of routing. Restoring the rule brings replies back, so the fault is isolated to that one rule.

Commands used on the instances

```
# find the private IP of the local instance
hostname -I

# tests 02 and 03: replace <peer-private-ip> with the other instance's private IP
ping -c 4 <peer-private-ip>

# test 04: SSH to the private instance by jumping through the administration host
ssh -i <key>.pem -J ec2-user@<admin-host-ip> ec2-user@<ewb-ec2-b-private-ip>

# test 05: confirm there is no public address from the AWS CLI
aws ec2 describe-instances --instance-ids i-0b2fa2f3b7306a262 i-08a657c4b1065e30b \
    --query 'Reservations[].Instances[].[InstanceId,PublicIpAddress,PrivateIpAddress]' --output table
```

What a successful ping looks like

A passing forward or reverse test ends with a summary like the one below. The values in angle brackets depend on the instances; the packet counts are what matter. For test 01 and the failure phase of test 06, the same command prints no reply lines and finishes with 100% packet loss.

```
$ ping -c 4 <peer-private-ip>
64 bytes from <peer-private-ip>: icmp_seq=1 ttl=<n> time=<x> ms
64 bytes from <peer-private-ip>: icmp_seq=2 ttl=<n> time=<x> ms
64 bytes from <peer-private-ip>: icmp_seq=3 ttl=<n> time=<x> ms
64 bytes from <peer-private-ip>: icmp_seq=4 ttl=<n> time=<x> ms
--- <peer-private-ip> ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
```

Optional extra checks

Check	Command	What it proves
Route table contents	`aws ec2 describe-route-tables --route-table-ids rtb-0a50e327a9e62155e`	The peer CIDR points at the pcx target and its state is "active", not "blackhole"
No internet from VPC B	`curl -m 5 https://example.com` on ewb-ec2-b	Times out: peering does not lend VPC A's internet gateway to VPC B
Packet path	`ip route` on either instance	The host only sees a default route to the VPC router; the peer CIDR is handled by the VPC route table, not by the host

7. Forward path and return path

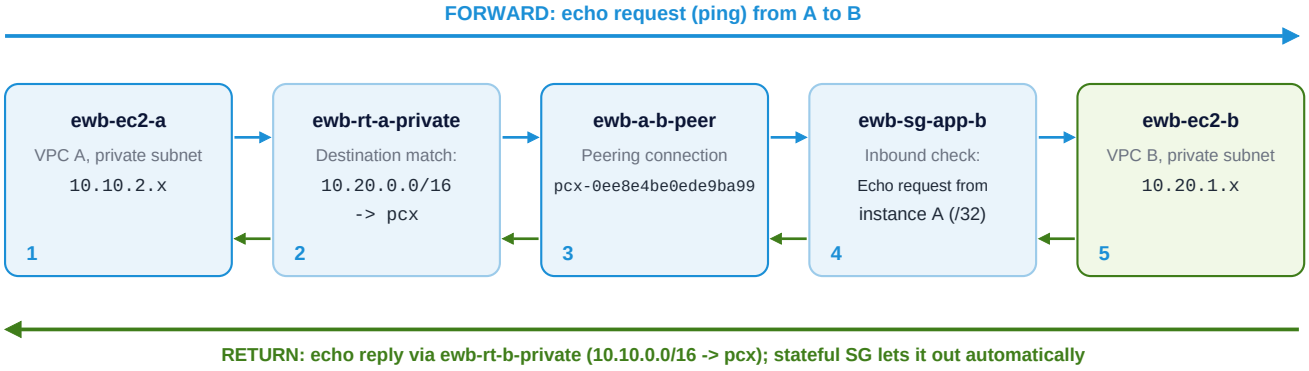


Diagram 4. Journey of one ping from ewb-ec2-a to ewb-ec2-b and back.

Forward path. A packet from the instance in VPC A to a 10.20.x.x address matches the 10.20.0.0/16 route in `ewb-rt-a-private`, whose target is the peering connection. It crosses the peering link and arrives in VPC B, where the destination security group must allow the traffic.

Return path. The reply to a 10.10.x.x address matches the 10.10.0.0/16 route in `ewb-rt-b-private`, which also points at the peering connection. Without that route the reply would have nowhere to go, so a one-way route is not enough. The security group is stateful, so the reply needs no extra inbound rule.

8. Key concepts and questions answered

- **Why is the return route needed?** Routing is per direction. If only VPC A knows the way to VPC B, requests arrive but replies cannot get back.
- **Why does an Active peering connection not guarantee a successful ping?** Active only means the link exists. Route tables, security groups and network ACLs still decide whether traffic is allowed.
- **Can VPC B use the internet gateway of VPC A over peering?** No. Peering is not transitive and does not extend internet access, so VPC B stays private.
- **Why must the CIDR ranges not overlap?** Overlapping ranges would make routing ambiguous, so AWS blocks the peering connection.
- **Is peering transitive?** No. A-B plus A-C does not give B-C connectivity.
- **Why must the request be accepted if both VPCs are in my account?** Acceptance is always a separate step. The same account simply performs both the request and the accept.
- **Does the peering connection itself have a security group?** No. Security is enforced by the security groups and network ACLs of the resources at each end.

9. Troubleshooting checklist

Symptom	What to check
Ping fails after acceptance	Both private route tables, the correct pcx ID and Active state, the Echo request rule on the destination, and the actual host IPs.
Peering creation fails	Check every CIDR of both VPCs for overlap, including secondary CIDRs.
SSH times out	Instance state, destination IP, subnet route table, then the security group rule, in that order.
Permission denied (publickey)	Correct key pair, local identity file and the ec2-user name. Routing is working if SSH authentication was reached.
Ping works in one direction only	The return route in the other VPC's route table, then the security group on the side that is not replying.
Peering stays in Pending acceptance	Accept it from the accepter side before the seven-day expiry; otherwise it expires and must be requested again.
Route shows status Blackhole	The peering connection it points to was deleted. Remove the stale route or create a new peering connection and update the target.
VPC B cannot reach the internet	Expected. VPC B has no internet gateway and peering never shares one.

10. Cost, quotas and limits

VPC peering is one of the cheapest ways to join networks because the connection itself carries no charge. What does cost money in this lab is the compute and storage around it.

Topic	What applies
Creating the peering connection	No charge to create or keep it.
Data transfer, same Availability Zone	Free over a peering connection. Both lab instances are in ap-southeast-2a.
Data transfer, across AZs in one Region	Billed at the standard in-Region data transfer rate, in each direction.
Data transfer, across Regions	Standard inter-Region data transfer rates apply.
Resources that do bill	EC2 instance hours, EBS volumes, and any public IPv4 address or Elastic IP. Terminate and delete them after the lab.
Active peering connections per VPC	Default quota 50, adjustable up to 125.
Outstanding peering requests	Default quota 25.
Unaccepted request lifetime	One week (168 hours), fixed; it then expires.

Check the current EC2 data transfer pricing page for exact rates before planning a production design, because prices vary by Region and change over time.

11. Choosing peering or an alternative

Peering is the right tool for a small number of VPCs that need direct, private access. Other services fit other shapes of network.

Option	Best for	Transitive routing	Things to know
VPC peering	A few VPCs that need simple private links	No	No hourly fee; CIDRs must not overlap; every pair needs its own connection and routes.
Transit Gateway	Many VPCs or hub-and-spoke designs	Yes, through the gateway	Hourly attachment and data processing charges apply.
AWS PrivateLink	Exposing one service privately to other VPCs	Not applicable	Works even when CIDRs overlap; clients reach only that service.
Site-to-Site VPN or Direct Connect	Linking an on-premises network to AWS	Depends on design	Used for hybrid connectivity rather than VPC to VPC links.

The number of connections grows quickly in a full mesh: n VPCs need n(n-1)/2 peering connections, so 100 VPCs would need 4,950. That is the usual reason teams move from peering to Transit Gateway as they grow.

12. AWS CLI equivalent

Everything in section 4 was done in the console. The commands below reproduce the same build from a terminal, which makes the lab repeatable. Shell variables hold the IDs returned by each call; `SG_B` and the host IPs are the values from your own security groups and instances.

VPC A, subnets and internet gateway

```
export AWS_DEFAULT_REGION=ap-southeast-2
VPC_A=$(aws ec2 create-vpc --cidr-block 10.10.0.0/16 --query Vpc.VpcId --output text)
aws ec2 create-tags --resources $VPC_A --tags Key=Name,Value=ewb-vpc-a
SUB_A_PUB=$(aws ec2 create-subnet --vpc-id $VPC_A --cidr-block 10.10.1.0/24 \
    --availability-zone ap-southeast-2a --query Subnet.SubnetId --output text)
SUB_A_PRIV=$(aws ec2 create-subnet --vpc-id $VPC_A --cidr-block 10.10.2.0/24 \
    --availability-zone ap-southeast-2a --query Subnet.SubnetId --output text)
IGW=$(aws ec2 create-internet-gateway --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC_A
```

Route tables in VPC A

```
RT_A_PUB=$(aws ec2 create-route-table --vpc-id $VPC_A --query RouteTable.RouteTableId --output text)
aws ec2 associate-route-table --route-table-id $RT_A_PUB --subnet-id $SUB_A_PUB
aws ec2 create-route --route-table-id $RT_A_PUB --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW
RT_A_PRIV=$(aws ec2 create-route-table --vpc-id $VPC_A --query RouteTable.RouteTableId --output text)
aws ec2 associate-route-table --route-table-id $RT_A_PRIV --subnet-id $SUB_A_PRIV
```

VPC B, its subnet and route table

```
VPC_B=$(aws ec2 create-vpc --cidr-block 10.20.0.0/16 --query Vpc.VpcId --output text)
aws ec2 create-tags --resources $VPC_B --tags Key=Name,Value=ewb-vpc-b
SUB_B=$(aws ec2 create-subnet --vpc-id $VPC_B --cidr-block 10.20.1.0/24 \
    --availability-zone ap-southeast-2a --query Subnet.SubnetId --output text)
RT_B=$(aws ec2 create-route-table --vpc-id $VPC_B --query RouteTable.RouteTableId --output text)
aws ec2 associate-route-table --route-table-id $RT_B --subnet-id $SUB_B
```

Peering connection and routes on both sides

```
PCX=$(aws ec2 create-vpc-peering-connection --vpc-id $VPC_A --peer-vpc-id $VPC_B \
    --query VpcPeeringConnection.VpcPeeringConnectionId --output text)
aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id $PCX

aws ec2 create-route --route-table-id $RT_A_PRIV --destination-cidr-block 10.20.0.0/16 \
    --vpc-peering-connection-id $PCX
aws ec2 create-route --route-table-id $RT_A_PUB  --destination-cidr-block 10.20.0.0/16 \
    --vpc-peering-connection-id $PCX
aws ec2 create-route --route-table-id $RT_B      --destination-cidr-block 10.10.0.0/16 \
    --vpc-peering-connection-id $PCX
```

Security group rules and the controlled failure test

```
ECHO='IpProtocol=icmp,FromPort=8,ToPort=-1,IpRanges=[{CidrIp=<ewb-ec2-a-private-ip>/32}]'
aws ec2 authorize-security-group-ingress --group-id $SG_B --ip-permissions "$ECHO"
aws ec2 authorize-security-group-ingress --group-id $SG_B \
    --ip-permissions 'IpProtocol=tcp,FromPort=22,ToPort=22,IpRanges=[{CidrIp=<admin-host-ip>/32}]'

# test 06: remove only the Echo rule, ping fails, then restore the same rule
aws ec2 revoke-security-group-ingress    --group-id $SG_B --ip-permissions "$ECHO"
aws ec2 authorize-security-group-ingress --group-id $SG_B --ip-permissions "$ECHO"
```

13. Hardening and next steps

These improvements go beyond the lab as built. They are the sensible next moves if the design were to carry real workloads.

- **Replace the bastion path.** AWS Systems Manager Session Manager or EC2 Instance Connect Endpoint reach private instances without an open SSH port or a jump host.
- **Narrow the routes.** Point the peer route at the specific peer subnet (for example 10.20.1.0/24) instead of the whole VPC range, so only the intended network is reachable.
- **Reference security groups.** Within one Region, a rule can name the peer VPC's security group as its source, which survives IP changes better than a /32.
- **Turn on VPC Flow Logs.** Accepted and rejected records show exactly which layer dropped a packet, which speeds up troubleshooting.
- **Enable DNS resolution over the peering.** Peering options can let each side resolve the other's private DNS names to private IPs.
- **Describe it as code.** CloudFormation or Terraform makes the build repeatable and reviewable, and cleanup becomes a single stack delete.
- **Tag everything.** A shared Project tag makes the final "nothing chargeable remains" check a single filter.

14. Cleanup plan

To avoid charges, all resources created for this lab should be removed once the evidence is saved:

- Terminate the EC2 instances and confirm their volumes are deleted. Release any Elastic IP and remove the unused key pair.
- Delete the peer routes and the peering connection ewb-a-b-peer.
- Delete the lab security groups, then the subnets and custom route tables.
- Detach and delete ewb-igw-a, then delete both VPCs.
- Verify with the project tag that nothing chargeable remains.

Order matters: a resource cannot be deleted while something still depends on it. The commands below follow a safe order.

```
aws ec2 terminate-instances --instance-ids i-0b2fa2f3b7306a262 i-08a657c4b1065e30b
# wait until both instances show "terminated", then continue

aws ec2 delete-route --route-table-id $RT_A_PRIV --destination-cidr-block 10.20.0.0/16
aws ec2 delete-route --route-table-id $RT_A_PUB  --destination-cidr-block 10.20.0.0/16
aws ec2 delete-route --route-table-id $RT_B      --destination-cidr-block 10.10.0.0/16
aws ec2 delete-vpc-peering-connection --vpc-peering-connection-id pcx-0ee8e4be0ede9ba99

aws ec2 delete-security-group --group-id $SG_A
aws ec2 delete-security-group --group-id $SG_B
aws ec2 delete-subnet --subnet-id $SUB_A_PUB
aws ec2 delete-subnet --subnet-id $SUB_A_PRIV
aws ec2 delete-subnet --subnet-id $SUB_B
aws ec2 delete-route-table --route-table-id $RT_A_PUB
aws ec2 delete-route-table --route-table-id $RT_A_PRIV
aws ec2 delete-route-table --route-table-id $RT_B
aws ec2 detach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC_A
aws ec2 delete-internet-gateway --internet-gateway-id $IGW
aws ec2 delete-vpc --vpc-id $VPC_A
aws ec2 delete-vpc --vpc-id $VPC_B
```

15. What I learned

This project showed me that a working network is the combination of several small pieces, and that each piece has one clear job. A VPC peering connection only creates the link. Routes on both sides send the traffic across it, and security groups decide which hosts may talk. I also learned why careful CIDR planning at the start matters, since overlapping ranges cannot be fixed later without rebuilding. Testing from both directions, and breaking one rule on purpose, made it much easier to understand where a connectivity problem can come from. The same ideas apply to many real AWS designs, such as connecting application and database environments privately.

Key takeaways

- Peering creates the link; routes, network ACLs and security groups decide what crosses it.
- Routes are needed in both directions, and the most specific route always wins.
- Peering is non-transitive and never shares an internet gateway.
- Test both directions and break one rule on purpose to find which layer is responsible.
- Clean up in dependency order to avoid leftover charges.

Appendix A: Glossary

Term	Meaning
VPC	Virtual Private Cloud: a logically isolated virtual network in an AWS Region.
CIDR	Classless Inter-Domain Routing notation for an address range, such as 10.10.0.0/16.
Subnet	A slice of a VPC's address range placed in one Availability Zone.
Route table	A set of rules that tells traffic leaving a subnet where to go next.
Internet gateway (IGW)	A VPC component that lets resources with public addresses reach, and be reached from, the internet.
Peering connection (pcx)	A private, non-transitive network link between two VPCs.
Security group	A stateful, instance-level firewall; inbound traffic is denied unless a rule allows it.
Network ACL	A stateless, subnet-level firewall; the default NACL allows all traffic.
Stateful	Replies to allowed traffic are permitted automatically without a matching rule.
Longest prefix match	The rule that makes the most specific matching route (largest prefix length) win.
Bastion / administration host	A hardened host used as the single entry point for administering private instances.
Blackhole route	A route whose target no longer exists, so matching traffic is dropped.

Appendix B: Further reading

- Amazon VPC Peering Guide
  `docs.aws.amazon.com/vpc/latest/peering/`
- VPC peering connection quotas
  `docs.aws.amazon.com/vpc/latest/peering/vpc-peering-connection-quotas.html`
- Building a Scalable and Secure Multi-VPC AWS Network Infrastructure (whitepaper), VPC peering section
  `docs.aws.amazon.com/whitepapers/latest/building-scalable-secure-multi-vpc-network-infrastructure/vpc-peering.html`